Individual Assignment

Module Code	CT070-3-3-DPAT Design Patterns
Intake Code	APD3F2511SE
Student Name	Youssef alaa shaker Abdelwahed hammad
TPNumber	TP076844

Table of contents

1. Introduction

In today's day and age, we have become dependent on software to look for information, connect with friends and family, book hospital appointments, study, or even perform payments. Since software has become such an integral part of our lives, software's quality has become a fundamental concern for the development and maintenance of the software. McCall's Quality model is one of the most influential frameworks that discuss quality factors that affect the developer developing the software and the user using the software.

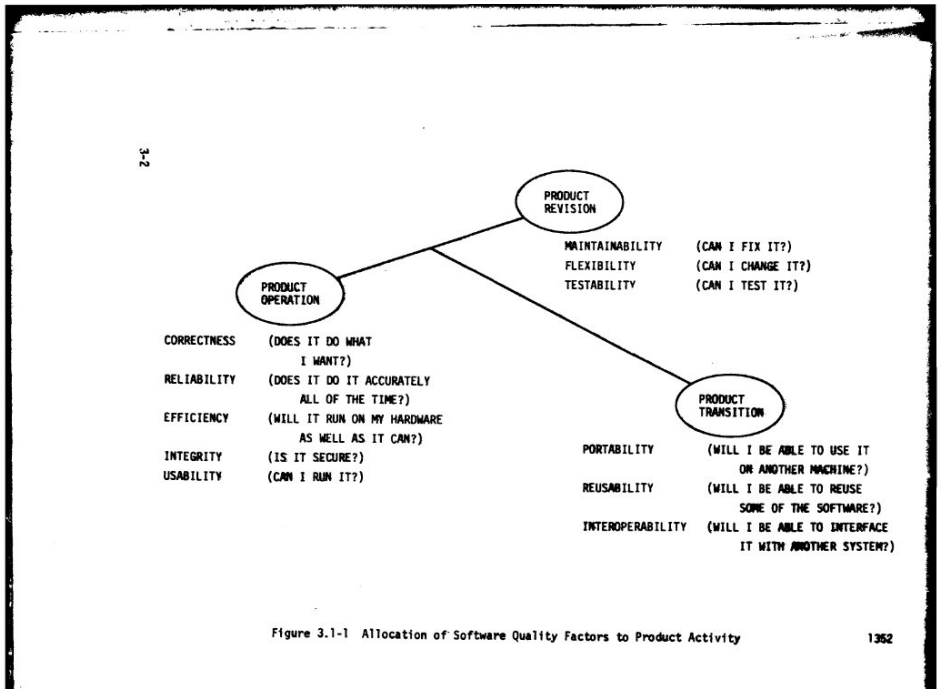

Figure 1.1: McCall's quality model taken from McCall et al. (1977)

McCall's quality model classifies all software requirements into 11 software quality factors which are classified into 3 product quality factors: Product Revision, which measures a software's Maintainability, Flexibility, and Testability; Product Operations which measures a software's correctness, reliability, efficiency, integrity, and usability; and finally product Transition, which measures a software's portability, interoperability, and re-usability (McCall et al., 1977).

As software systems increase in scale and complexity, recurring design problems and structural patterns tend to emerge across different components and features. Addressing these recurrences through code duplication is widely regarded as an unsustainable practice, as it introduces

inconsistencies, increases maintenance overhead, and significantly reduces long-term development efficiency. Re-usability, therefore, serves as a critical quality attribute that enables developers to leverage existing, well-tested components in new contexts, thereby streamlining the development process and improving the overall integrity of the software system.

This case study aims study through empirical evidence the effects design patterns has on the re-usability of a software. The software through which this empirical study will be performed is a newspaper subscription notification system. This paper will compare an implementation with design patterns to an implementation without design patterns in terms of code re-usability.

2. Software specifications

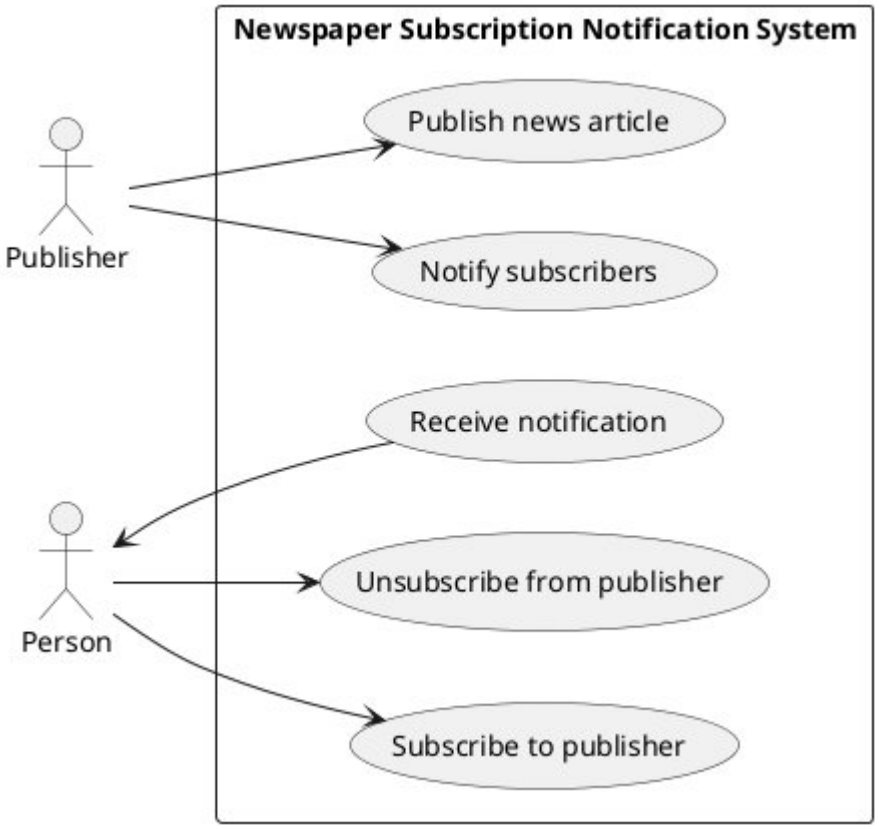

Figure 2.1: use-case diagram showing the use cases of the system

The system developed in this report is a Newspaper Subscription Notification System, in which multiple newspaper publishers notify their subscribers whenever a new newspaper comes out. The flow of the system should be as follows (figure 2.1 demonstrates the use cases):

1. Person subscribes to a newspaper provider to get updates from that provider.
2. Person chooses what notification channel to receive the updates on (i.e push-notifications, SMS alerts, or email notifications)
3. Newspaper gets updated
4. Newspaper updates its subscribers that a new newspaper has been released.

3. Solutions

3.1. Solution 1: No design patterns

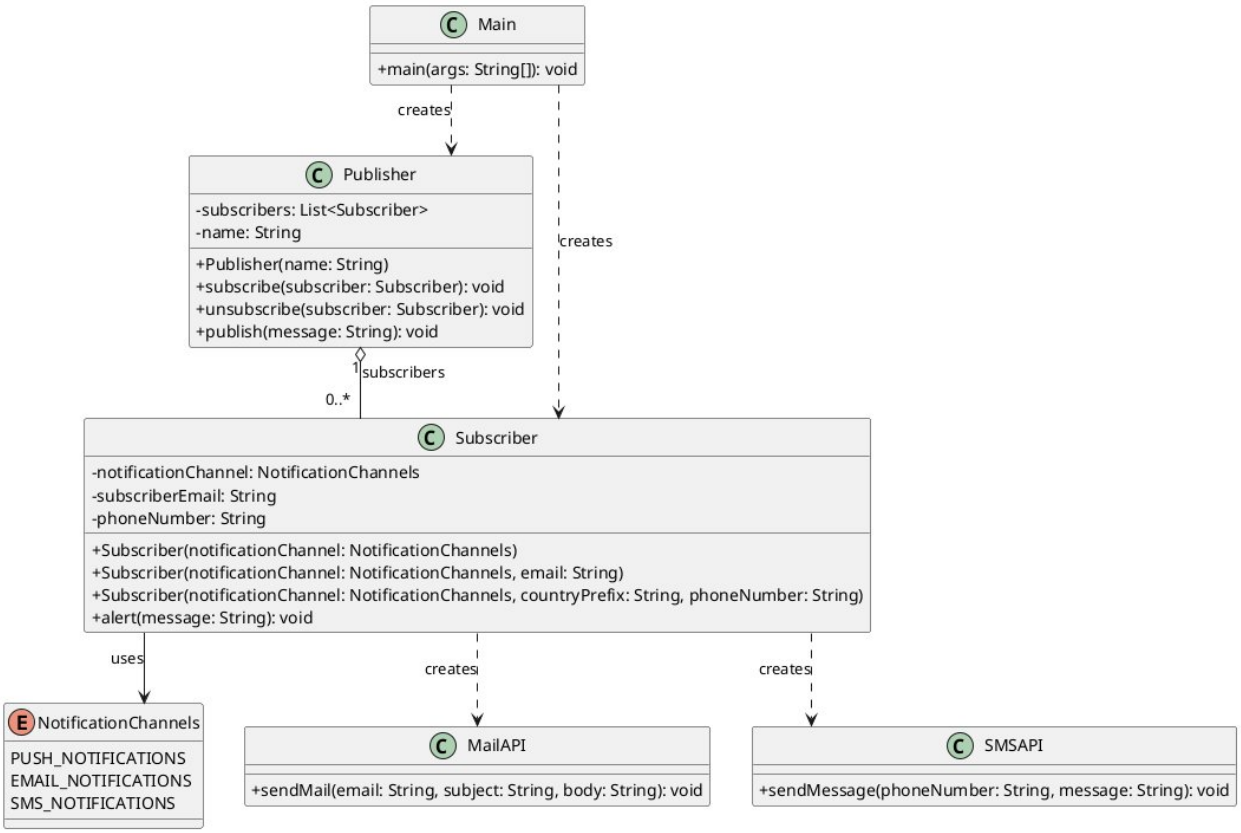

Figure 3: UML for the solution without design patterns

Figure 3 shows the UML diagram for the solution without employing design patterns. The given solution works fine, however many parts of the code are not reusable, which causes inefficiency when adding new features.

If for instance we need to use the same mechanism to send notifications to subscribers about their favorite TV shows, a new class, similar to the publisher class would have to be added:

Figure 4: new publisher class is replicating many of the functions of the Publisher class

Since there is no interface or an abstract class to dictate how the publishing class would look like, the code is replicated. Also, the subscribe, unsubscribe, and publish methods are the same for both classes:

```
public void subscribe(Subscriber subscriber) {  5 usages
    subscribers.add(subscriber);
}

public void unsubscribe(Subscriber subscriber) {  1 usage
    subscribers.remove(subscriber);
}

public void publish(String message) {  1 usage
    for (Subscriber subscriber : subscribers) {
            subscriber.alert(message);
    }
}
```

Figure 5: Code for one of the publishing classes.

Another problem we will run into, is how each one will be used in the main class. Since there is no interface or an abstract class, the user of the publisher class would need to use the class name directly:

```
Publisher newYorkTimes = new Publisher( name: "new york times");
```

Figure 6: Publisher class being used in the main method

This lack of polymorphism causes the code to be repetitive where it should be reusable.

The notification logic employed in solution is all implemented inside the Subscriber.alert(Message) method:

```
public void alert(String message) {  1 usage
    if (this.notificationChannel == NotificationChannels.PUSH_NOTIFICATIONS) {
        System.out.println("You have a new push notification from the news outlet: " + message);
    } else if (this.notificationChannel == NotificationChannels.EMAIL_NOTIFICATIONS) {
        MailAPI mailInstance = new MailAPI();
        mailInstance.sendMail(subscriberEmail,  subject: "news", message);
    } else if (this.notificationChannel == NotificationChannels.SMS_NOTIFICATIONS) {
        SMSAPI smsInstance = new SMSAPI();
        smsInstance.sendMessage(phoneNumber, message);
    }
    System.out.println();
}
```

Figure 7: Implementation for each notification channel.

Because everything is in the same class, adding new logic for the notifications will necesitate the programmer to add more code to the Subscriber class, which would eventually cause the Subscriber class to be bloated and unreadable. Also, we can see that the subscriber class has three different constructors, each catering to a different notification mechanism: the second constructor for instance, just assigns the email, and the thrid one assigns the user's phone number. Finally the email and phone number fields are useless if one is used without the other (if the user chooses the email notificationChannel, then the phone number field will not be utilized, and if they use the phone number field, the email will not be utilized.

3.2. Solution with Design patterns:

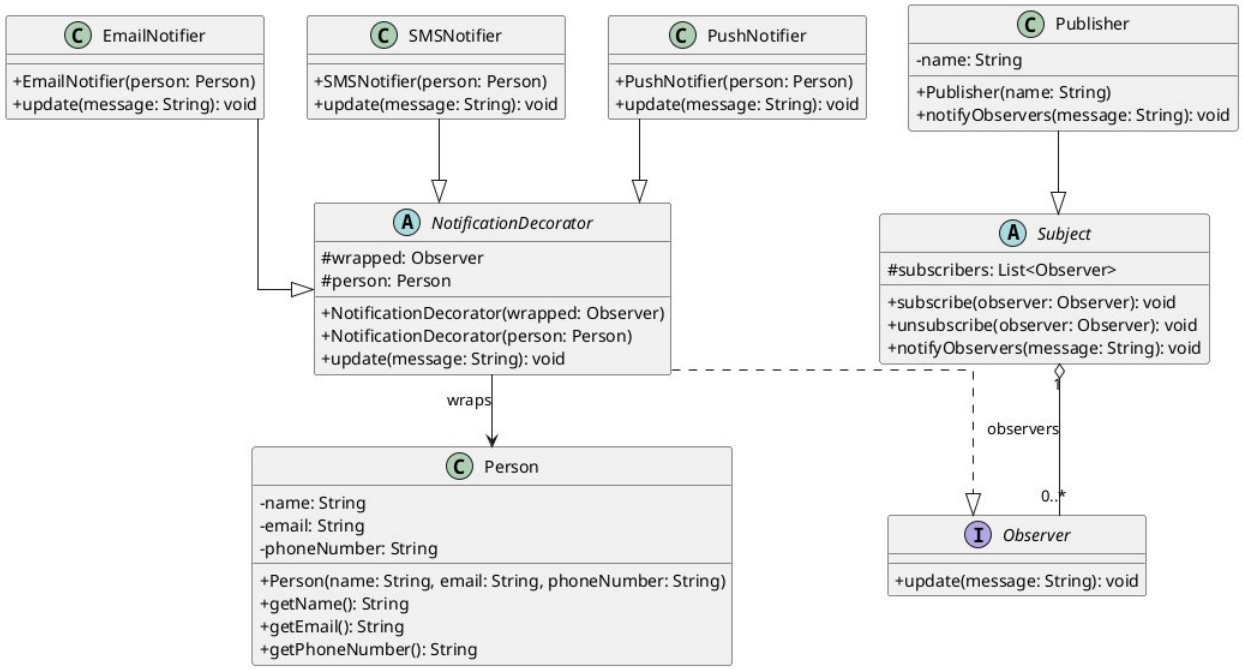

Figure 8: Decorator and Observer implemented

In the old implementation, the Observer (subscriber) implemented the notification implementation in the Subscriber class. That implementation had the issue of expanding everytime a new method of notification would be added to the program. In this implementation, the NotificationDecorator encapsulates the behaviour of sending notifications to the Subscriber, and the person class (previously the subscriber) is just there to represent the person subscribing to the publisher. The NotificationDecorator and its children implement the Observer interface in order for them to get notified every time the publisher publishes a new thing.

The Observer pattern was implemented in order for the notification mechanisms to get notified everytime a new thing comes out. The Person class does not get notified, instead, to add a layer of reusability, and not have to create multiple Subscriber objects each with a different constructor just to subscribe to different notification channels, the Person class is wrapped with a notifier or multiple notifiers in order to facilitate sending the notification on one or several channels. Note that the Publisher only needs to override the notifyObservers method in order to send the message to the subscribers.

By implementing the observer pattern, we also allow for more entities to be plugged into the system, like the TVShowProducer:

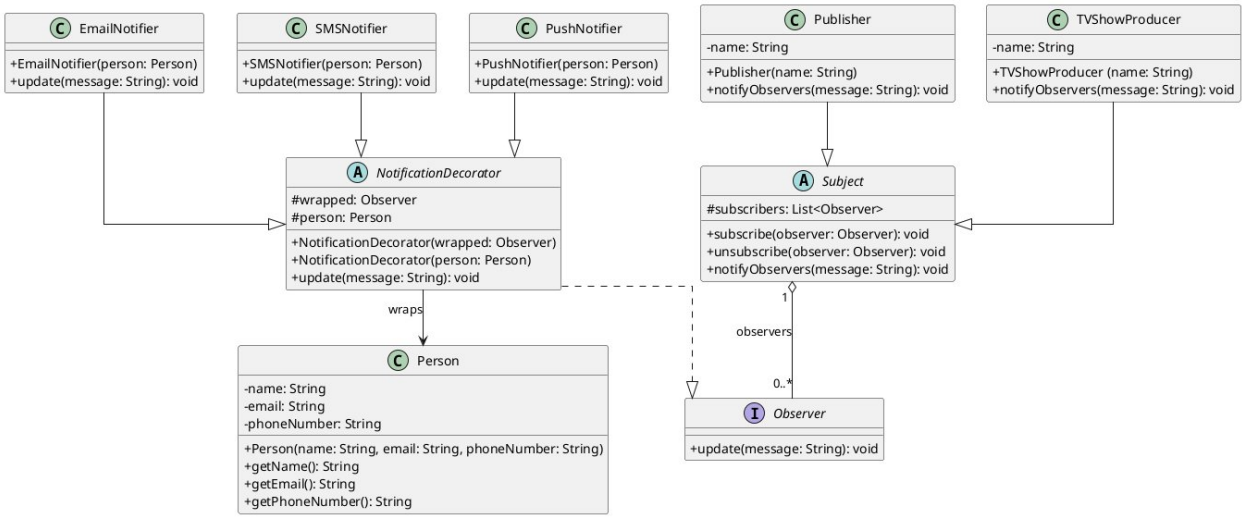

Figure 9: TVShowProducer added

The TVShowProducer class only needs to inherit the Subject abstract class and override the notifyObservers method to send a different message, rather than having to repeat all the other implementations which are the same.

4. Empirical Evaluation

This section presents an empirical evaluation of two alternative software design solutions developed for the same application: a simpler object-oriented design without the use of design patterns (S1), and a refined design incorporating appropriate design patterns (S2). The purpose of this evaluation is to determine which design better supports the selected software quality attribute, namely reusability, as defined in McCall's Software Quality Model.

Reusability refers to the extent to which software components can be used in other applications or contexts with minimal modification. Improving reusability is a key objective in software engineering, as it reduces development time, increases maintainability, and promotes consistency across systems. Design patterns, as formalized in Gamma et al. (1994), are widely regarded as effective solutions for enhancing software modularity and promoting reuse through well-structured and decoupled designs.

In order to objectively compare S1 and S2, this evaluation adopts a set of established object-oriented metrics to quantify reusability. These metrics provide measurable indicators of design quality, allowing for a systematic comparison between the two approaches. The results obtained from this analysis will be used to determine whether the application of design patterns leads to a demonstrable improvement in reusability, thereby supporting informed design decisions.

4.1. Measuring Techniques for Reusability

Evaluating software reusability requires the use of quantitative metrics that reflect the structural quality of a system's design. As this evaluation is conducted using UML class diagrams rather than implementation-level data, the assessment focuses on design-level characteristics such as coupling, cohesion, and modularity. These characteristics are widely recognized as key indicators of reusability, as they determine how easily individual components can be isolated, understood, and reused in different contexts. To ensure a systematic and objective comparison between the simpler solution (S1) and the design pattern-based solution (S2), a set of established object-oriented metrics is applied. These metrics enable the derivation of measurable values from the UML diagrams, forming the basis for empirical comparison.

4.1.1. Selected Metrics

- Coupling Between Objects (CBO):

Measures the number of dependencies between classes by analyzing associations and relationships in the UML diagram. Lower coupling indicates fewer dependencies, which improves the ability to reuse classes independently.

- Lack of Cohesion in Methods (LCOM):

Evaluates how closely related the responsibilities within a class are, based on its attributes and methods. Lower LCOM (higher cohesion) suggests that a class has a well-defined purpose, making it easier to reuse.

4.2. Justification of Metrics

The selected metrics are based on the CK metrics suite proposed by Chidamber and Kemerer (1994). Coupling and cohesion are fundamental design principles that directly influence reusability, since tightly-coupled or poorly structured classes are difficult to extract and reuse in other systems.

4.3. Analysis

The selected metrics were applied to both design solutions, S1 (non-design pattern solution) and S2 (design pattern-based solution), by analyzing their respective UML class diagrams. The measurements were derived by examining class relationships, dependencies, and structural organization within each diagram. The results are summarized in Table 1.

Class (Without Patterns)	**LCOM Score**	**Class (With Patterns)**	**LCOM Score**

Subscriber	High	**Concrete Notifiers**	Low
NotificationChannels	N/A	**NotificationDecorator**	Low

Class (Without Patterns)	**CBO**	**Class (With Patterns)**	**CBO**	**Analysis**
Subscriber	4	EmailNotifier	1	The original Subscriber is tightly coupled to MailAPI, SMSAPI, and NotificationChannels. The EmailNotifier only depends on its parent decorator.
Publisher	1	Subject	1	Both are coupled to their respective observer/subscriber types, but the Pattern version uses an interface (Observer), making it more flexible.
Main	3	Main (Implicit)	1	In the "Without" version, Main must know how to create the Subscriber and all its dependencies. In the Pattern version, Main only needs the concrete notifier.

5. Conclusion

The Tables in chapter 4 shows that the design patters solution has Low lack of cohesion methods, showing that most classes have one and only one responsibility, which makes the code reusable, also, the second table, shows that the classes with patterns showed a lower Copuling between objects (CBO) which shows that the classes are losely coupled, and that they can be reused in other systems.

6. References

Chidamber, S., & Kemerer, C. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, *20*(6), 476–493. https://doi.org/10.1109/32.295895

Gamma, E., Helm, R. F., Johnson, R. E., & Vlissides, J. (1994). *Design patterns: elements of reusable Object-Oriented software*. http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=016722060&sequence=000002&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA

McCall, J., Richards, P., & Walters, G. (1977). *Factors in software quality: Concept and Definitions of software quality* [PDF]. ROME AIR DEVELOPMENT CENTER. https://apps.dtic.mil/sti/tr/pdf/ADA049014.pdf